Formative 1 - Data Quality and Performance

In this formative, I have outlined my understanding of key data engineering concepts, including data monitoring, database normalisation, data quality management and data ingestion methods. I have applied these concepts to realistic workplace scenarios and explained their importance in delivering reliable, high-quality data solutions.

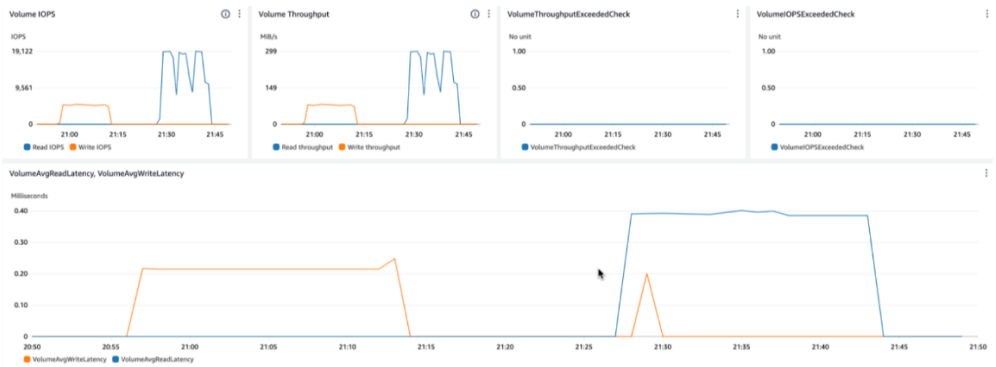
1. Monitoring Data Stores (K1, S7)

In my organisation, I work primarily with data stores across AWS, Google Cloud Platform (GCP) and on-premise PostgreSQL databases. Effective monitoring is essential to ensure data remains available, performant, and aligned with agreed service levels. I use monitoring tools that provide real-time metrics, dashboards, and alerting, enabling issues to be identified and resolved before they impact downstream users.

For AWS, I primarily use Amazon CloudWatch. A key metric I monitor is read/write latency, as increased latency can indicate throttling, inefficient queries, or resource contention. When latency exceeds the agreed threshold, I review logs and workload patterns before taking corrective actions such as query optimisation, capacity scaling, or workload redistribution.

For our on-premise PostgreSQL databases, we integrate database monitoring with a program called Dynatrace. A key metric I monitor is database response time, as deterioration can indicate inefficient queries, locking issues, resource constraints, or increased workload demand. Alerts are set up and when response times rise above expected levels, I can then investigate database statistics, identify the root cause, and work with the relevant teams to optimise queries or address infrastructure issues.

For GCP, I use Cloud Monitoring and track request error rates as an indicator of service health and availability. Prompt investigation of alerts helps minimise downtime and maintain reliable data products for downstream consumers.



Evidence 1 – Showing Amazon CloudWatch monitoring of latency & throughput. There is a limit of 32,000 IOPS and 4,000 MiB/s, with an alert set if the application breaches these limits in 3 of the last 5 mins.

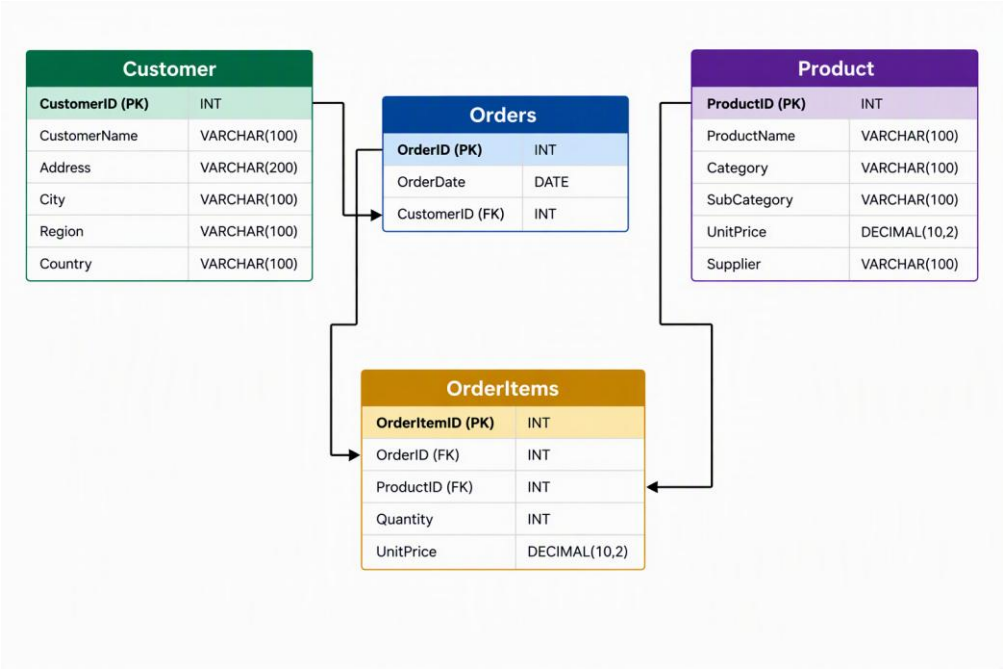
2. Normalisation Principles (K3)

To achieve **First Normal Form (1NF),** I would ensure each field contains a single value and remove repeating groups. In a messy Orders table, I would create one row per product within each order. Assuming each product appears once per order, OrderID and ProductID would together identify each row.

To achieve **Second Normal Form (2NF),** I would remove partial dependencies, where information depends on only part of this composite key. I would move order details into an Orders table and product details into a Products table. An OrderItems table would link orders and products, storing information such as quantity that depends on both keys. This reduces duplication and prevents inconsistent updates.

To achieve **Third Normal Form (3NF),** I would move customer names and addresses from Orders into a Customers table keyed by CustomerID, retaining CustomerID in Orders as a foreign key. This removes transitive dependencies because customer details depend on CustomerID rather than directly on OrderID.

This is important because normalisation reduces data redundancy, improves data quality, and makes databases easier to maintain. For example, if a customer has placed 100 orders and changes their address, an unnormalised database may require 100 separate updates. With a normalised structure, the address only needs to be updated once in the Customers table, ensuring consistency and reducing the risk of errors across the dataset.



Evidence 2 – I created this Entity-Relationship Diagram of the normalised Orders relational database that I described in this task (Key: PK – Primary Key, FK - Foreign Key).

3. Data Quality & Risk (K5)

A risk at the ingestion stage is receiving **incomplete source data**, such as a CSV file with missing records or a failed PostgreSQL extract. This could result in inaccurate reports, failed reconciliations, or incomplete downstream datasets. **I mitigate this risk by** using data profiling checks that validate row counts, mandatory fields, and expected file arrivals. **I surface issues through** pipeline monitoring and Dynatrace alerts so that failed loads can be quarantined and investigated before the data is published.

A risk at the transformation stage is **schema drift**, where an upstream system changes a column name, data type, or structure without notice. This can cause pipeline failures or incorrectly transformed data. **I mitigate this risk by** implementing schema validation checks and maintaining version-controlled data contracts. **I surface issues through** automated validation failures and monitoring alerts, allowing changes to be reviewed before they affect production data.

A risk at the data sourcing stage is using data without **verified ownership**, consent, or usage permissions. This could lead to regulatory breaches, ethical concerns, or reputational damage. **I mitigate this risk by** maintaining audit trails that record data ownership, approval status, and intended use. **I surface issues through** governance reviews and access-control exceptions, ensuring that unapproved data cannot be ingested.

In summary, data quality risks can occur at every stage of the data lifecycle. By implementing controls and identifying issues early, I can prevent unreliable or inappropriate data from entering data systems. This is important because high-quality data supports accurate reporting, trusted analytics, and informed business decisions, which are key responsibilities of a data engineer.

```python
import json
from pathlib import Path

import requests
from jsonschema import Draft202012Validator, FormatChecker

SCHEMA = json.loads(Path("user.schema.json").read_text())
Draft202012Validator.check_schema(SCHEMA)          # fail fast on a broken schema
VALIDATOR = Draft202012Validator(SCHEMA, format_checker=FormatChecker())


def test_user_matches_schema():
    resp = requests.get("https://jsonplaceholder.typicode.com/users/1", timeout=10)
    assert resp.status_code == 200

    errors = list(VALIDATOR.iter_errors(resp.json()))
    assert not errors, "\n".join(
        f"{e.json_path}: {e.message}" for e in errors)
```

Evidence 3 – I used Python to validate JSON data against a JSON schema to detect schema drift.

.

4. Ingestion Frameworks (K18, S15)

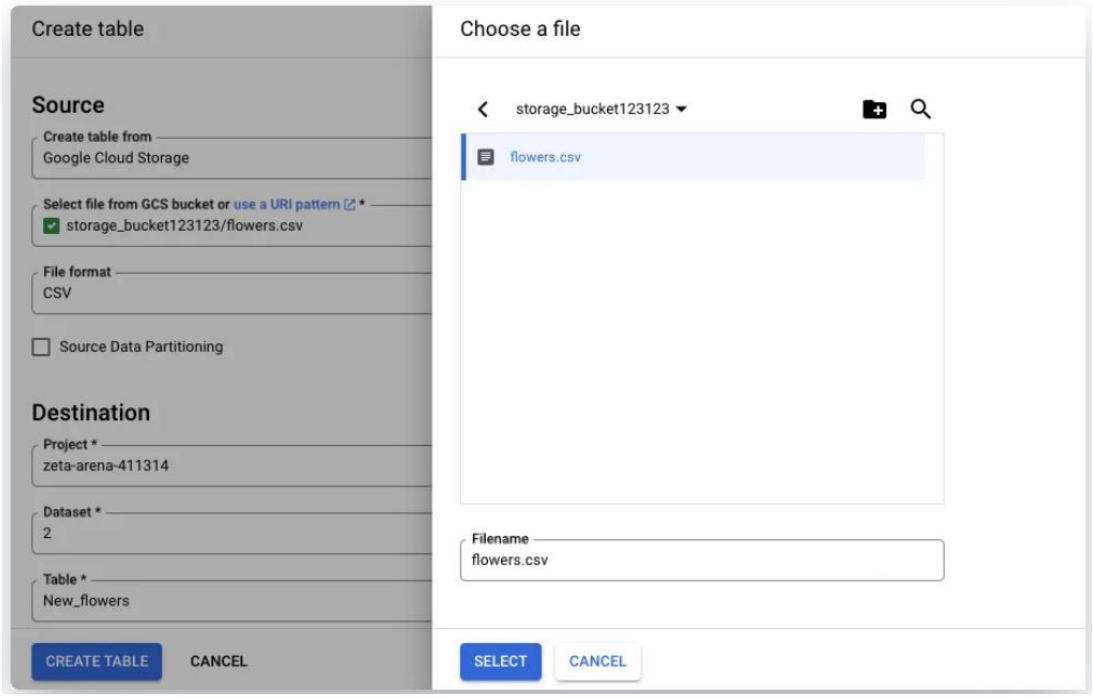
In my role, I use **batch ingestion** to move raw data from Google Cloud Storage (GCS) into BigQuery for analytics and reporting.

A **performance implication** of this approach is that large files can increase load times and query costs if the data is not structured efficiently. For example, loading several months of raw data into a single unpartitioned table can result in BigQuery scanning significantly more data than required for a query.

To **optimise the ingestion process,** I use partitioning and clustering within BigQuery so that queries only scan the relevant subset of data. I also load data in bulk rather than processing individual records, which improves throughput and reduces storage required. Where possible, I apply schema validation and data quality checks during ingestion to ensure only trusted data reaches analytical datasets.

This is important because efficient ingestion processes reduce processing time, lower storage and query costs, and improve the performance of downstream reporting and analytics.

In summary, selecting the correct ingestion method and applying optimisations such as partitioning, clustering, and bulk loading helps ensure data is moved reliably and remains accessible for business users and data analysts.



Evidence 4 – Example (using sample data) of batch ingesting data (.csv file) from GCS to BigQuery in the Google Cloud Platform.

Conclusion

In conclusion, effective monitoring, data quality controls, normalised data structures, and optimised ingestion processes are all essential aspects of data engineering. By applying these principles in a practical context, I can help ensure data remains accurate, accessible, and valuable for business reporting, analytics, and decision-making.

General